INTERVIEW PREPARATION

Senior DevOps Engineer Ansible & On-Prem CI/CD

Realtime and tricky scenario Q&A covering CI/CD pipeline design, Ansible & configuration management, on-premises DevOps, DevSecOps, Kubernetes/ OpenShift, observability, and cross-team release coordination across UK & Poland stakeholders.

91
QUESTIONS & ANSWERS

17
TOPIC AREAS

Prepared for interview practice — treat behavioral/leadership answers as frameworks to personalize with real examples.

Contents

Ansible	15 questions
On-Prem DevOps	6 questions
CI/CD Pipelines	9 questions
Git & Branching	5 questions
Maven & Build	4 questions
DevSecOps	5 questions
Docker	5 questions
Kubernetes & OpenShift	7 questions
Scripting	5 questions
Observability	6 questions
Artifactory & Nexus	2 questions
Cloud AWS/Azure	3 questions
Release Management	3 questions
Enterprise Infra	4 questions
PostgreSQL & Unix	4 questions
Tricky Scenarios	5 questions
Communication & Leadership	3 questions

Ansible

Core Concepts

Q1. Walk me through how Ansible actually works under the hood — no agents, so how does it execute anything on a remote host?

Ansible is agentless and push-based. The control node connects to the managed node over SSH (or WinRM for Windows), copies small Python modules to a temp directory on the target, executes them, captures JSON output, and cleans up — unless you use ansible-pull, where the target pulls its own playbook from a repo instead. That's the core distinction interviewers are checking for: no persistent daemon, no polling loop like Chef/Puppet agents. The control node needs Python (or the target does, for module execution) and SSH access with sudo/become rights. I'd add that inventory (static INI/YAML or dynamic, e.g. aws_ec2 plugin) defines what 'remote host' even means, and facts gathering (setup module) runs first unless gather_facts is disabled — which matters for performance at scale.

Q2. Is Ansible idempotent by default, or is that something you have to build in? TRICKY

Trick in the question — it's neither fully automatic nor something you always have to hand-build. Ansible's built-in modules (yum, apt, file, copy, template, user, service, etc.) are idempotent by design: they check current state before acting and report 'changed: false' if nothing needs to happen. But the moment you drop into shell or command modules, idempotency is entirely on you — those run every time regardless of state, which is exactly why linting and code review catch raw shell tasks without a 'creates:' or 'when:' guard. I always flag shell/command tasks in review for exactly this reason, and prefer creates=, removes=, or changed_when overrides to make them idempotent explicitly.

Q3. A playbook run shows 40 hosts as 'ok' and 2 as 'changed' on a run you expected to be a complete no-op. How do you investigate, live, on a call? TRICKY REAL-TIME

First I don't panic-assume drift — I check what module those 2 tasks touched with -vv or --diff, since diff mode shows exactly what changed (file content, template render, package version). Common real causes: a fact that varies by host (hostname baked into a template, a timestamp, an IP-dependent block), a config-management race where another process modified the file outside Ansible, or genuinely different package/OS versions across those 2 hosts making a version-pinned package task actually install something. I'd run --check --diff against just those 2 hosts with -l to isolate, compare to the other 40, and only then decide if it's expected drift correction (good) or a template bug leaking host-specific data (bug). I'd also check if those 2 hosts have a different group_vars/host_vars override that's legitimately different — sometimes 'changed' is completely correct and the other 38 were already converged from a prior run.

Playbooks & Roles

Q1. How do you structure a large Ansible codebase so it doesn't become unmaintainable across dozens of environments and teams?

Roles are the unit of reuse — one role per logical component (nginx, postgres, app-deploy), each with its own tasks/handlers/templates/defaults/vars/meta, following ansible-galaxy's standard directory layout. I keep environment-specific data out of roles entirely and push it into inventory: group_vars/all.yml for global defaults, then group_vars/<env>.yml and host_vars/<host>.yml for overrides, so the same role runs unchanged in dev, UAT, and prod with only variable values differing. For real scale I'd also split site.yml into smaller playbooks per component and use ansible.cfg + roles_path to pull shared roles from a private Galaxy/Artifactory-hosted collection rather than copy-pasting roles between repos — that's exactly the kind of on-prem tooling this role is asking about.

Q2. You need to restart a service only if a config file actually changed, but only once even if three different tasks touch that file. How?

TRICKY REAL-TIME

Handlers, triggered with notify=, and Ansible automatically deduplicates — if three tasks all notify 'restart nginx' in the same play, the handler fires exactly once, at the end of the play (or immediately if you call meta: flush_handlers). The tricky part people miss: handlers only run if the notifying task reports changed, and they run in the order they're *defined* in the handlers file, not the order they were notified — that ordering detail has bitten people when they assume notify order controls execution order. I'd also mention listen: for grouping multiple handler names under one trigger name when several roles need to react to the same event, like a generic 'restart web services' listen topic.

```
- name: deploy nginx config
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: restart nginx

handlers:
  - name: restart nginx
    service:
      name: nginx
      state: restarted
```

Q3. What's the difference between vars, defaults, and group_vars/host_vars, and what's the actual precedence order?

defaults/main.yml inside a role is the lowest precedence — easily overridden, meant as sane fallback values. vars/main.yml inside a role is higher precedence, meant for values the role author doesn't expect you to casually override. group_vars and host_vars live in inventory and sit above role vars but below play-level vars and extra-vars (-e on the CLI, which always wins). The full official precedence order runs roughly: role defaults → inventory group_vars/all → inventory group_vars/<group> → inventory host_vars → playbook group_vars/host_vars → host facts → play vars → role vars → block vars → task vars → extra-vars. In practice I keep it simple: defaults for role authors, group_vars/host_vars for environment differences, -e reserved for one-off CLI overrides during troubleshooting, never for anything that should be checked into source control.

Q4. Real scenario: a playbook that's supposed to be safe to re-run keeps failing on step 12 of 20 during a deploy window, and you're under pressure to just get it done. What's your actual move?

TRICKY REAL-TIME

I resist the urge to just re-run blindly and instead run with --start-at-task to resume from step 12 once I understand why it failed — but only after confirming steps 1-11 are genuinely idempotent, because if any of them aren't, resuming skips state they'd have fixed. I'd check --list-tasks first to see the exact task name, then look at the actual error rather than assume. If it's a transient issue (network blip, package mirror timeout) I'd just retry that task. If it's a real config problem, I fix the role and do a full --check --diff dry run before re-applying for real. I'd also mention serial: in the play — if this were a rolling deploy across many hosts and step 12 failed on host 3 of 50, serial with max_fail_percentage would have already stopped the blast radius rather than me discovering it mid-deploy across all hosts at once.

Q5. When would you reach for a role versus a collection versus just a flat playbook?

Flat playbook for something genuinely one-off or environment-specific that won't be reused. A role once logic needs to be reused across playbooks or projects — it's the natural reuse boundary in Ansible. A collection once you're packaging multiple related roles, modules, plugins, or filters together for distribution — typically when a platform team is publishing shared building blocks to the rest of the org via Galaxy or an internal Artifactory/Nexus-backed collection registry, which is exactly the kind of on-prem tooling maturity this JD is pointing at. I've drawn that line before on platform teams: individual app teams write playbooks and lightweight roles; the platform team owns and versions the shared collection everyone consumes.

Vault & Secrets

Q1. A junior engineer commits a playbook with a plaintext database password to Git by mistake. What's your real, full response — not just 'use Vault'? TRICKY REAL-TIME

Immediate: rotate the credential — a git revert doesn't remove it from history, it's already compromised the moment it's pushed, so credential rotation is priority one, before anything about the repo. Then I'd scrub history with git filter-repo or BFG if the repo is shared/long-lived, force-push, and notify anyone who may have already pulled. Root cause fix: ansible-vault encrypt the variable file (or per-variable with !vault inline encryption via ansible-vault encrypt_string, which is cleaner for mixed files since only the secret is encrypted, not the whole file), wire the vault password through a vault password file the CI/CD system pulls from a proper secrets store (HashiCorp Vault, Azure Key Vault, AWS Secrets Manager) rather than a static file on disk, and add a pre-commit hook or gitleaks/trufflehog scan in the pipeline so this class of mistake gets caught before merge, not after. I'd treat the incident itself with a short blameless writeup, because the process gap — no secret-scanning gate — is the actual fix, not the individual mistake.

Q2. How do you manage multiple Vault passwords for different environments without it becoming a mess?

ansible-vault supports multiple vault IDs, so I'd label them explicitly — dev, uat, prod — each with its own password source, and reference vault_identity_list in ansible.cfg or --vault-id on the CLI. Practically: --vault-id prod@prompt for anything sensitive enough to require a human to type it interactively during a prod run, and --vault-id dev@/path/to/script for lower environments where a script can pull it from a secrets manager non-interactively. The real discipline is never letting one password decrypt every environment's secrets — a leaked dev vault password should never be able to touch prod.

```
ansible-vault encrypt_string 'S3cr3t' --vault-id prod@prompt --name 'db_password'
ansible-playbook site.yml --vault-id dev@~/.vault_pass_dev --vault-id prod@prompt
```

Inventory & Scale

Q1. On-prem, no cloud dynamic inventory plugin to lean on — 400 physical/VM hosts, inventory drifts constantly as hosts get added/decommissioned. How do you keep it accurate? TRICKY REAL-TIME

This is exactly the on-prem gap cloud-native tooling assumes away. My approach: build a lightweight dynamic inventory script (Python, inventory plugin interface) that sources truth from whatever CMDB or asset system already exists on-prem — could be a simple database, ServiceNow, or even a well-maintained CSV/spreadsheet if that's the org's reality — rather than trusting a hand-edited static INI file that inevitably goes stale. I'd also add a scheduled job that diffs the inventory script's output against actual reachable hosts (a quick ansible all -m ping -i dynamic_inventory.py) and alerts on hosts that are in inventory but unreachable, or reachable-but-unmanaged hosts that never got added — that mismatch is usually where decommission and provisioning process gaps actually live. If there's genuinely no CMDB, I'd push for standing one up even minimally, because untracked inventory is the single biggest source of 'why didn't this patch apply everywhere' incidents I've seen.

Q2. How do you control blast radius when rolling out a change to hundreds of on-prem servers?

serial: is the primary lever — serial: 10 or serial: '20%' processes hosts in batches rather than all-at-once, and combined with max_fail_percentage, the play aborts if failures exceed a threshold instead of grinding through every remaining host in a broken batch. For genuinely risky changes I'll do serial: 1 first against a canary host, validate, then widen. I also lean on tags and --limit heavily during rollout — limiting to a specific group first (a canary group in inventory) before touching the full fleet, and any_errors_fatal: true when a failure on any host in a batch should stop the whole play rather than continue degrading other hosts in parallel.

```
- hosts: web_fleet
  serial:
    - 1
    - 10
    - "25%"
  max_fail_percentage: 10
```

Troubleshooting

Q1. ansible-playbook hangs indefinitely on one host with no error, no timeout, nothing in -vvv that explains it. What are your hypotheses, in order? TRICKY REAL-TIME

First hypothesis: it's not Ansible, it's SSH — a stuck TTY prompt (host key verification, sudo password prompt with become and no password configured, or an interactive script being called by a shell/command task) will hang exactly like this with no useful Ansible-level error. Second: a task genuinely waiting — a service start that's blocking, a package manager lock held by another process on that host (yum/apt lock file from a manual admin session), or a wait_for task with too generous a timeout. Third, less common but real: ControlPersist SSH multiplexing socket getting into a bad state from a prior interrupted run. My actual live steps: Ctrl+C and check if it's genuinely stuck at a specific task via -vvv output before killing, SSH to that one host manually and check for lock files (/var/lib/dpkg/lock, /var/run/yum.pid), check ps aux for a hung process from the previous task, and check ~/.ansible/cp/ for a stale control socket to remove.

Q2. Difference between --check mode and --diff, and where does --check mode actually lie to you?

--check runs a dry-run — no changes applied, Ansible reports what it would have done. --diff shows the actual before/after content difference for file-based modules (template, copy, lineinfile), and the two are almost always used together: --check --diff. Where --check lies: any task with a when: condition that depends on state --check itself would have created gets evaluated against the *unchanged* real state, so multi-step plays where task 2 depends on task 1's actual effect can report misleadingly — task 2 might show 'skipped' in check mode even though it would run for real, because task 1 never really executed to satisfy its condition. Modules that genuinely can't simulate (most shell/command tasks, some third-party modules without check_mode support) either skip entirely or, worse, some poorly-written custom modules ignore check_mode and actually execute — which is a real gotcha to test for before trusting --check output on anything with custom modules in the mix.

Enterprise Ansible

Q1. What does Ansible Automation Platform (formerly Tower/AWX) add over raw ansible-playbook that matters for an enterprise on-prem environment?

RBAC and audit trail — who ran what playbook against what inventory, when, is a hard compliance requirement most enterprises need and raw CLI Ansible has none of. Centralized credential storage so SSH keys and vault passwords never live on individual engineers' laptops. Scheduling, a REST API for integrating playbook runs into other tooling (like a Jenkins pipeline triggering a Job Template rather than shelling out to ansible-playbook directly), surveys for parameterized self-service runs, and workflow templates for chaining multiple playbooks with conditional branching. For a UK/Poland cross-team setup specifically, the RBAC and audit layer is usually the actual business driver — it's what lets you delegate 'restart this specific service' to a support team without giving them SSH keys or full playbook authorship rights.

On-Prem DevOps

Fundamentals

Q1. What actually changes in your day-to-day approach going from cloud-native DevOps to on-premises DevOps? TRICKY

The honest answer: you lose elasticity as a safety net. In cloud, undersizing a VM or running out of disk is often a one-line fix and a few minutes of reprovisioning; on-prem, capacity is physical and lead-time-bound — a rack running hot on storage means a procurement conversation, not an API call. So capacity planning becomes a real, ongoing discipline rather than something autoscaling papers over. Second, there's no managed-service safety net — no RDS, no managed Kubernetes control plane; you own patching the hypervisor, the OS, the database engine, the load balancer firmware, end to end. Third, network and physical topology matter again — VLANs, physical NICs, SAN/NAS paths, rack-level redundancy — in a way cloud abstracts away almost entirely. What doesn't change: the CI/CD, IaC, and automation discipline should look the same; the goal on-prem is still to make infrastructure changes repeatable and version-controlled rather than click-ops on a hypervisor console, it's just harder to get there because the tooling ecosystem is thinner.

Q2. How do you approach infrastructure-as-code on-prem when you don't have a cloud provider's API to target?

Terraform actually has solid on-prem providers — vSphere, libvirt/KVM, and bare-metal providers for things like PXE-boot provisioning, plus providers for on-prem network gear (F5, NetApp, Cisco). Where there's no clean provider, Ansible fills the gap for configuration and even some provisioning via modules that talk to iLO/iDRAC (Dell) or IPMI directly for bare-metal power/boot control. My usual on-prem pattern: Terraform (or a vSphere/vCenter-specific tool) for the VM/resource-pool layer where a provider exists, Ansible for everything above that — OS config, application deployment, patching — and PXE + kickstart/preseed or a golden-image pipeline (Packer building on-prem templates) for bare-metal or fresh-VM OS provisioning. The key discipline is the same as cloud: nothing gets changed by logging into vCenter or a server console by hand if it can be codified instead.

Q3. A production database server on-prem is out of disk space at 2am and paging you. Cloud instinct says 'resize the volume.' What's actually different here? TRICKY REAL-TIME

On-prem, resizing usually means extending a LUN on the SAN (if there's headroom in the storage pool) or attaching additional physical/virtual disk, then extending the filesystem and volume group live — LVM makes this non-disruptive if done right (pvresize, vgextend, lvextend, then resize2fs or xfs_growfs depending on filesystem). But the actual root-cause step matters more here than in cloud: I'd first check what's actually consuming space — logs, a runaway temp table, orphaned WAL/binlog files — because on-prem storage isn't 'just add more' the way cloud volumes are; the storage team may not have unlimited headroom in that pool, and I don't want to burn scarce physical capacity plugging a leak that a log-rotation fix would have solved for free. Immediate mitigation (clear safe-to-delete logs, rotate/compress), then the LVM extend if genuinely needed, then a post-incident fix — log rotation policy, disk-usage alerting at 80% not 95%, and a written runbook so this isn't a 2am judgment call next time.

Environment Management

Q1. How do you keep dev/UAT/prod environments consistent on-prem when you can't just spin up identical cloud accounts per environment?

Golden images built once (Packer or a scripted kickstart/preseed process) and promoted through environments rather than hand-built per environment — that's the single biggest lever for consistency. Same Ansible roles and Terraform modules applied to every environment, with only variable values (group_vars per environment) differing, never different playbook logic per environment — divergent logic per environment is exactly how 'works in UAT, breaks in prod' happens. I'd also push for environment parity checks — a scheduled job that diffs installed package versions, config file checksums, and running service versions across environments and flags drift, since on-prem environments drift silently in a way ephemeral cloud environments structurally can't.

Q2. UAT has been passing for weeks, but the moment a release hits prod it fails — and prod and UAT are supposedly identical. How do you actually find the difference? TRICKY REAL-TIME

I don't trust 'supposedly identical' — I verify it. Concretely: diff installed package versions (rpm -qa or dpkg -l piped to diff between environments), diff config files that Ansible manages versus what's actually on disk (ansible-playbook --check --diff against prod tells you if prod has drifted from the same playbook that built UAT), check for data-volume differences that only manifest at scale (a query that's fine against UAT's small dataset but times out against prod's real volume), and check for environment-specific config that's supposed to differ but might have a wrong value — a connection pool size, a feature flag, a DNS/hosts entry pointing somewhere stale. In my experience the actual culprit is disproportionately often either a silently-drifted config file that was hand-edited on prod once during an incident and never reconciled back into the playbook, or a data-scale issue UAT's smaller dataset never exercised — so I check both early rather than assuming it's a code bug.

Middleware & App Servers

Q1. On-prem environments often still run traditional app servers alongside newer containerized workloads — what's your approach to bridging both worlds in CI/CD?

I treat them as two deployment targets behind the same pipeline concept, not two separate pipelines with different quality bars. Build and test stages (Maven build, SonarQube scan, unit/integration tests) stay identical regardless of target. The deployment stage branches: for containerized services, build and push an image, then a Kubernetes/OpenShift deploy step; for traditional middleware (Tomcat, WebLogic, JBoss), the pipeline packages a WAR/EAR and uses Ansible to handle the deploy — stop service, backup current deployment, drop new artifact, start service, health check, with the same rollback discipline (keep last-known-good artifact) as the container path has via image tags. The goal is that from a release-management view, both paths look like one release train with the same gates — code quality, security scan, approval — even though the last-mile mechanics differ.

CI/CD Pipelines

Pipeline Design

Q1. Walk me through how you'd design a CI/CD pipeline from commit to production for a typical enterprise Java service, on-prem-friendly, using tools in this JD.

Commit triggers the pipeline (Jenkins or Azure DevOps). Stage 1: build with Maven, running unit tests as part of the build lifecycle, failing fast on compile or test failure. Stage 2: static analysis — SonarQube quality gate, plus a SAST scan; the pipeline fails the build if the quality gate fails, not just reports it. Stage 3: package — build the artifact, and if containerized, build and tag the Docker image; push both the Maven artifact to Nexus/Artifactory and the image to an internal registry. Stage 4: deploy to a lower environment automatically (dev or UAT), run integration/ smoke tests, and if it's a security-sensitive service, a DAST scan against that running environment. Stage 5: promotion to prod is typically gated — manual approval tied to a change record, especially in a regulated or enterprise on-prem shop — then deploy via the same artifact that was tested (never rebuild for prod), with a rollback plan (previous artifact version retained in Nexus/registry, deployment strategy that supports quick rollback). I'd instrument the whole pipeline so failures are visible fast and each stage's duration is tracked, because pipeline speed itself becomes a productivity metric worth protecting.

Q2. A pipeline goes green — all stages pass, artifact deployed — but the application is actually broken in production. How do you explain that, and what do you change? TRICKY REAL-TIME

Green means 'the checks that exist passed,' not 'the application works' — those are different claims, and the gap is always a missing or weak check. I'd walk backward: did the smoke test after deploy actually assert anything meaningful, or just check that the process started? Did integration tests run against realistic data, or a trivial fixture that doesn't exercise the actual failure path? Was there a config difference between the tested environment and prod that the pipeline never validated — a secret, a connection string, a feature flag? Concretely I've seen this be a health check that only checked TCP port open, not application-level readiness; I'd fix that first, since it's usually the cheapest, highest-value gap. Structural fix: add a genuine post-deploy smoke test that exercises a real user-facing path, add config validation as an explicit pipeline stage rather than trusting it implicitly, and make sure rollback is automatic on smoke-test failure rather than requiring a human to notice.

Q3. How do you handle secrets in a CI/CD pipeline so they're never in plaintext in a Jenkinsfile or pipeline YAML?

Never inline — always pulled at runtime from a secrets store. In Jenkins, the Credentials plugin with withCredentials binds a secret to an environment variable only for the scope of that block, and Jenkins masks it in console output automatically. In Azure DevOps, variable groups linked to an Azure Key Vault, or secret variables marked explicitly secret so they're masked in logs. Either way, the actual secret value should live in Vault/Key Vault/a secrets manager, not in pipeline definition files, not in a Jenkins global variable visible to every job, and definitely not in a Groovy script committed to source control. I'd also scope credential access narrowly — a deploy job for service A shouldn't have blanket access to every secret in the org's vault, just the ones that service needs.

```
withCredentials([usernamePassword(credentialsId: 'db-cred', usernameVariable: 'DB_USER', passwordVariable: 'DB_PASS')]) {
  sh 'deploy.sh --user=$DB_USER --pass=$DB_PASS'
}
```

Jenkins & Groovy

Q1. Declarative versus scripted Jenkins pipelines — when do you actually reach for scripted?

Declarative is my default — structured, readable, has built-in stage visualization, and enforces enough guardrails that most engineers on a team can read and modify it without deep Groovy knowledge, which matters for maintainability across a team spanning UK and Poland where consistency reduces handoff friction. I drop into scripted, or a script{} block inside declarative, when I need genuine programmatic logic — dynamic stage generation based on a discovered list of services in a monorepo, complex conditional branching that declarative's when: directive can't express cleanly, or custom error handling with try/catch/finally around specific steps. In practice the pattern I like best is declarative pipeline as the outer structure, with a script{} block for the genuinely complex bits, rather than going fully scripted — keeps 90% of the pipeline approachable.

Q2. Your Jenkins shared library function works perfectly when you test it standalone, but fails mysteriously only when called from inside a parallel block in the actual pipeline. What's going on? TRICKY REAL-TIME

Classic Groovy/Jenkins CPS (Continuation-Passing-Style) transformation issue combined with shared context in parallel execution. Two usual suspects: first, closures capturing a shared/mutable variable across parallel branches — if your shared library function reads or writes a variable defined outside the parallel block without proper scoping, branches can race on it, since Jenkins parallel steps genuinely run concurrently, not just interleaved. Second, CPS serialization — Jenkins pipeline scripts get transformed to support pause/resume, and certain Groovy constructs (some closures, non-serializable objects) behave differently under CPS than in plain Groovy, which is exactly why it works standalone but not inside the pipeline's CPS-transformed execution. My fix approach: make sure any state the function touches is passed as an explicit parameter, not captured from outer scope, add @NonCPS on methods that don't need to survive a pipeline restart and are pure computation, and test the shared library function specifically inside a parallel block in a scratch pipeline before trusting it in the real one.

Q3. How do you structure Jenkins shared libraries so multiple teams (say, UK and Poland squads) reuse pipeline logic without stepping on each other?

One shared library repo with versioned tags (@v1.2.0 in the @Library annotation, not @master, so teams pin a known-good version and aren't broken by someone else's in-flight change), vars/ for global pipeline steps callable directly, src/ for the actual Groovy classes with real logic and unit-testable structure. I'd organize functions by capability, not by team — a deployToKubernetes() step, a runSonarScan() step — so both teams call the same well-tested building blocks with different parameters, rather than each team maintaining a forked copy. Governance-wise, changes to the shared library go through their own PR review and a version bump, exactly like a published package, and I'd version-pin per-project Jenkinsfiles so an update to the library doesn't silently break every pipeline across both teams simultaneously.

Deployment Strategies

Q1. You're doing a blue-green deployment on-prem — no cloud load balancer auto-provisioning. After cutover, half the traffic is still hitting the old version 20 minutes later. Why, and what do you check? TRICKY REAL-TIME

On-prem, blue-green cutover is only as instant as whatever's routing traffic, and that's rarely as clean as a cloud load balancer's API-driven target-group swap. Usual suspects: DNS-based cutover with a TTL that hasn't expired yet — if cutover relies on a DNS record change, clients and resolvers caching the old TTL will keep hitting blue for however long that TTL says, which is exactly why DNS-based blue-green needs a short TTL set well in advance of cutover, not at cutover time. Second: a hardware or software load balancer (F5, HAProxy, nginx) where the pool/member config change didn't actually propagate — some on-prem LBs need an explicit config reload or have connection draining that keeps existing sessions pinned to the old pool intentionally. Third: sticky sessions/session affinity keeping already-connected users on blue by design, which is correct behavior, not a bug, if that's how the LB is configured. My actual check order: confirm the LB config itself shows green as active, check connection draining settings, check DNS TTL and whether resolvers are honoring it, and distinguish 'new connections still going to blue' (real bug) from 'existing sessions still on blue' (expected, will drain naturally).

Q2. Canary versus blue-green versus rolling — how do you decide which one for a given service?

Blue-green when you need instant, clean rollback and can afford to run two full environments simultaneously even briefly — good for services where a bad deploy is expensive and you want zero ambiguity about which version is live. Canary when you want to validate a change against real production traffic at small scale before committing fully — route 5% of traffic, watch error rates and latency against a baseline, then progressively widen; best when you have real metrics and alerting tied to the canary cohort specifically, otherwise you're just guessing. Rolling (the Kubernetes/OpenShift default) when you don't need the strong guarantees of the other two and want resource efficiency — no need to double capacity, and it's simpler to reason about for most routine deploys. On-prem specifically, blue-green is often the hardest to justify because it doubles the hardware/licensing footprint during cutover, which matters more when you're not paying cloud's pay-as-you-go model — so I've leaned toward rolling with health-check gating as the default and reserved blue-green for genuinely high-risk releases.

Pipeline Reliability

Q1. What's the failure mode where a pipeline step reports success but the shell script it ran actually failed? TRICKY

Classic pipefail gap — in a pipeline where a shell step runs a command through a pipe (cmd1 | cmd2), the exit code of the whole pipeline defaults to the exit code of the *last* command, so if cmd1 fails but cmd2 (say, grep or tee) succeeds, the shell reports success even though the real work failed. Same failure shape with a multi-line shell block where an earlier command fails but the script keeps running and the last line happens to succeed — without set -e, bash doesn't stop on error by default. Fix: set -euo pipefail at the top of every shell script/step — -e stops on first error, -u catches unset variable references, -o pipefail propagates failure through pipes instead of masking it. I've had this exact bug in a real pipeline where a deployment step piped output through tee for logging, the actual deploy command failed, and the pipeline happily reported success because tee itself succeeded.

Git & Branching

Strategy Design

Q1. GitFlow versus trunk-based development — how do you actually decide which fits a given team, rather than just having a favorite?

It comes down to release cadence and team size/maturity, not personal preference. GitFlow (develop, feature branches, release branches, hotfix branches) fits teams with scheduled, less-frequent releases, multiple versions in production simultaneously needing parallel maintenance (common in enterprise/regulated on-prem shops with formal release trains), or teams still building CI/CD maturity where long-lived branches give more review/staging time before merge. Trunk-based (short-lived feature branches, frequent merges to main, feature flags instead of long-lived branches) fits teams doing genuine continuous delivery, strong automated test coverage, and comfort with feature flags to decouple deploy from release. Given this JD mentions coordinating releases across UK and Poland teams with formal release management, I'd lean toward a GitFlow-influenced model — release branches specifically help there, since they give a stabilization window before a coordinated multi-team release — but I'd push for shorter-lived feature branches within it and avoid GitFlow's worst failure mode, which is develop and feature branches living for months and turning every merge into a conflict-resolution project.

Q2. A hotfix needs to go to prod today, but prod is running release branch v2.3 while develop already has three sprints of unreleased work on top. How do you branch and merge this without polluting develop with half-finished features? TRICKY REAL-TIME

Branch the hotfix directly off the prod tag/release branch (v2.3), not off develop — that's the entire point of GitFlow's hotfix branch pattern. Fix it there, test it in isolation against exactly what's in prod, then merge the hotfix in two directions: into the release/main branch so prod actually gets it, tagged as v2.3.1, and separately into develop (or the current in-flight release branch if one exists) so the fix doesn't get silently lost the next time develop ships — that second merge is the step people forget under pressure, and it's how the same bug reappears two releases later. If develop has diverged significantly, I'd resolve conflicts at merge time rather than trying to cherry-pick the fix forward blind, and I'd tag the hotfix commit clearly so the release notes and audit trail show exactly what shipped outside the normal release train — that traceability matters for release management sign-off.

Q3. What's your actual branch protection and merge policy for a shared main/develop branch on an enterprise repo?

Required PR review (at minimum one approval, two for anything touching shared infra/security-sensitive code), required status checks passing before merge — build, tests, SonarQube quality gate, SAST — so a red pipeline physically can't merge, not just gets flagged. No direct pushes to main/develop, even for admins, enforced at the platform level, not just by convention — 'even for admins' is the part that actually matters, because an admin-bypass exception is exactly where unreviewed changes sneak into prod-adjacent branches. Require branches to be up to date before merge to catch integration conflicts early, and I'd require signed commits or at least verified authorship on anything touching the release/hotfix branches for audit purposes in a regulated on-prem environment.

Git Mechanics

Q1. You need to remove a large binary file accidentally committed three months and 200 commits ago — it's bloating every clone. git rm won't fix it. What do you actually do, and what's the risk? TRICKY REAL-TIME

git rm only removes it going forward — the blob stays in history, so every clone still pulls that 3-month-old commit and its full object. To actually purge it you need to rewrite history — git filter-repo (the modern, faster, safer replacement for the old filter-branch and BFG) targeting that specific path across all commits, which rewrites every commit SHA after that point. The real risk, and what I'd flag loudly before doing it: this rewrites history for everyone — every existing clone, every open branch, every open PR based on commits after that point becomes invalid and needs to be re-cloned or rebased onto the new history, so it needs to be coordinated as a scheduled, communicated event, not a quiet fix, especially across a multi-team UK/Poland repo where people are mid-feature-branch. I'd force-push the rewritten history, immediately notify every contributor to re-clone rather than pull/rebase (rebasing onto rewritten history from a stale local copy is where things go really wrong), and I'd also set up a pre-commit hook or Git LFS going forward so this specific mistake — large binaries in a normal repo — can't recur.

```
git filter-repo --path path/to/large-file.zip --invert-paths
git push --force --all
git push --force --tags
```

Q2. Rebase versus merge — what's your team guidance, and where do you draw the line?

For keeping a feature branch current with main/develop during development, I prefer rebase — it keeps history linear and readable, avoids noise merge commits, and makes bisecting failures much easier later. For actually bringing a completed feature branch into main/develop, I prefer a merge commit (often via PR merge with a no-fast-forward merge) because it preserves the fact that a feature branch existed as a unit, which matters for traceability and for reverting an entire feature cleanly if needed — reverting a squashed or rebase-merged single commit is much cleaner for that than untangling a rebased-in string of individual commits. The one hard rule regardless of preference: never rebase a branch that others have already pulled and built work on top of, since it rewrites SHAs and breaks everyone else's history — that's a coordination problem waiting to happen on a shared repo.

Maven & Build

Core Maven

Q1. Explain the Maven build lifecycle and where CI/CD hooks typically plug in.

Maven's default lifecycle runs through phases in strict order: validate, compile, test, package, verify, install, deploy — running any phase runs every phase before it. CI typically hooks in at 'test' or 'package' for the build-and-test stage — mvn clean test for fast feedback, mvn clean package to actually produce the artifact (JAR/WAR). SonarQube integration usually runs as a plugin bound to the verify phase, or as a separate mvn sonar:sonar goal right after package so it scans the built classes and test coverage together. 'install' puts the artifact in the local repo (mostly useful for local dev/multi-module builds), and 'deploy' pushes it to a remote repository manager — which is where Nexus/Artifactory comes in, via the distributionManagement config in the POM, typically the actual last CI step before the artifact is considered a real, promotable release candidate.

Q2. A build that's been stable for months suddenly fails in CI with a dependency resolution error, but works fine on your local machine. What's your triage? TRICKY REAL-TIME

First suspect: a transitive dependency version isn't actually pinned, and something upstream published a new version that CI resolved differently than your local machine's already-cached local repo — this is exactly why unpinned version ranges or SNAPSHOT dependencies are dangerous in a shared build. I'd check mvn dependency:tree on both environments and diff them. Second suspect: CI is pulling from a different repository mirror or has a stale/broken connection to Nexus/Artifactory — a repo manager outage or a proxy misconfiguration can surface as 'dependency not found' even though the dependency itself hasn't changed. Third: local machine has a stale cached copy of a dependency in ~/.m2 that no longer matches what's actually published — meaning local isn't actually 'working,' it's working against outdated state and would fail too with a clean cache. My fix priority: pin exact versions (no ranges, no SNAPSHOT in anything that ships), use a Nexus/Artifactory-hosted repo as the single source of truth with local dev pointed at the same mirror CI uses, and never let 'works on my machine' be the tiebreaker — reproduce clean with mvn clean -U (force update) to settle which environment is actually right.

Q3. How do you handle multi-module Maven projects in a CI pipeline without every module rebuilding on every commit?

Reactor build order handles the dependency graph automatically — mvn -pl <module> -am builds just the specified module and its dependencies (am = also-make), rather than the whole reactor. For real CI efficiency I'd combine that with change detection — most CI systems (or a simple git diff against the last successful build) can tell you which modules actually changed, and you build/test only the affected module plus anything depending on it. I'd also lean on Maven's incremental compilation and a good local/CI-shared cache (either the CI system's native caching or a shared Nexus-backed artifact cache) so unchanged modules resolve their dependencies fast rather than re-downloading. For genuinely large monorepos this is where I'd also evaluate whether Maven's reactor is even the right tool versus splitting into independently-versioned, independently-released modules with their own pipelines — reactor builds don't scale indefinitely.

Artifact Versioning

Q1. Why is a SNAPSHOT dependency dangerous in a production build pipeline, even if it 'usually works'? TRICKY

A SNAPSHOT version is explicitly mutable — the same version string, e.g. 1.2.0-SNAPSHOT, can resolve to genuinely different bytes today versus tomorrow, because it gets republished as work continues. That breaks the core guarantee a build pipeline depends on — reproducibility. A build that passed yesterday against 1.2.0-SNAPSHOT can fail (or worse, silently behave differently) today against the exact same POM, because the dependency itself changed underneath it without any change to your own code. In a release pipeline this is a real audit and rollback problem too — if you need to rebuild v3.4 six months from now to patch a security issue, and it depended on a SNAPSHOT, you may not even be able to reconstruct what actually shipped. My rule: SNAPSHOTs are fine for active in-progress inter-module development within a single team's sprint, but nothing that reaches a release branch, a tagged build, or gets promoted toward prod should ever depend on a SNAPSHOT — release versions get pinned exact versions, full stop.

DevSecOps

Static & Dynamic Analysis

Q1. Explain the difference between SAST and DAST, and where each fits in the pipeline.

SAST (Static Application Security Testing) analyzes source code without running it — looks for known-bad patterns, injection risks, hardcoded secrets, insecure API usage — and runs early, typically right after build, because it's fast and doesn't need a running environment; SonarQube's security rules, plus dedicated SAST tools, both live here. DAST (Dynamic Application Security Testing) tests a genuinely running application from the outside — probing actual HTTP endpoints for things like injection, auth bypass, misconfigured headers — which means it has to run later in the pipeline, against a deployed instance in a test/staging environment, because it needs something live to attack. The two catch different classes of issues and neither replaces the other: SAST catches code-level flaws before they're even deployed, DAST catches runtime/configuration flaws SAST structurally can't see (a header misconfiguration, an exposed endpoint that only exists once deployed). I'd gate merges on SAST/SonarQube (fast, blocks bad code early) and gate promotion-to-prod on DAST results against the UAT deploy (slower, but catches what SAST can't).

Q2. SonarQube's quality gate is blocking a release with a bug that the team genuinely believes is a false positive, and there's real pressure to ship today. What do you actually do? TRICKY REAL-TIME

I don't just override the gate to make the deadline — that erodes the entire point of having a gate, and it's exactly the kind of shortcut that becomes normal after the third time. Real process: verify it's actually a false positive, not a real issue someone's rationalizing away under deadline pressure — read the actual finding, not just the summary. If it's genuinely a false positive (SonarQube does have real false-positive rates on certain rule categories, especially around null-pointer analysis in code with runtime guarantees the static analyzer can't see), mark it as 'Won't Fix' or 'False Positive' in SonarQube itself with a documented justification, which is the sanctioned mechanism for exactly this situation — it keeps the audit trail intact rather than silently disabling the gate. If it's genuinely a real issue but low severity and the business accepts the risk, that's a documented risk-acceptance decision made by someone with the authority to accept it, not an engineering override — I'd escalate to whoever owns that call rather than making it myself under pressure. What I won't do is quietly lower the quality gate threshold or disable the rule org-wide just to unblock one release — that fix reaches every future build, not just today's.

Q3. How do you tune a SonarQube quality gate so it's meaningfully strict without generating so much noise that developers start ignoring it?

Start from the 'Sonar way' default gate but actually tailor it to the codebase's real state rather than either accepting defaults blindly or disabling everything inconvenient. Key levers: apply strict gates (zero new bugs/vulnerabilities, coverage threshold) to *new code* specifically, using SonarQube's new-code-period concept, rather than demanding an old, large legacy codebase retroactively meet a strict bar overnight — that's the single biggest thing that makes gates practical instead of demoralizing on brownfield projects. Set coverage thresholds based on genuinely achievable targets per project maturity rather than one blanket number across every repo. Review the rule set periodically with the team — if a rule is consistently flagged as false-positive-prone for legitimate reasons in your specific stack, that's a real signal to retune the rule, not just suppress individual findings forever.

Container & Dependency Security

Q1. A container image that passed vulnerability scanning at build time is now flagged critical two weeks later in a scheduled re-scan, with nothing in your code having changed. How is that possible, and what do you do?

TRICKY REAL-TIME

Nothing in your code changed, but the base image or an OS-level package inside it didn't stay frozen in the vulnerability database's eyes — a CVE gets disclosed after your build (this happens constantly; a package that was 'clean' at build time gets a new CVE published against a version you're already running), or your scanner's vulnerability database itself got updated with newly disclosed CVEs that retroactively apply to packages already baked into your image. This is exactly why 'scanned once at build time' isn't sufficient for container security — images sitting in a registry, or worse, already deployed and running, need periodic re-scanning against the current CVE database, not just a point-in-time check. My response: triage severity and actual exploitability first — not every critical CVE is exploitable in your specific deployment context (is the vulnerable code path even reachable given how you use that package?) — then patch by rebuilding from an updated base image and redeploying, following the same pipeline as any other release rather than a manual hotfix to a running container. Structurally I'd make sure registry-level scanning re-scans images on a schedule, not just at push time, and that a critical finding on a deployed image has an actual alerting/ticketing hook, not just a dashboard nobody checks.

Q2. Where in the pipeline do you actually put container image scanning, and what do you do when it finds something?

Two gates, not one: at build time, right after the image is built and before it's pushed to the registry — fail the push on critical/high findings above an agreed threshold, so a genuinely bad image never even lands in the registry. And on a recurring schedule against images already in the registry and already deployed, since new CVEs get disclosed against old images constantly, as above. Tooling-wise this integrates naturally into the pipeline as a stage right after docker build, before docker push — Trivy, Grype, or a registry-native scanner (Artifactory Xray, Harbor's built-in scanner) all fit that slot. On a finding: severity and reachability triage first, patch by bumping the base image or the specific vulnerable package and rebuilding — never patch a running container in place and call it done, since the next deploy from the same unpatched image definition just reintroduces the vulnerability.

Docker

Image Building

Q1. Your Docker image build takes 12 minutes in CI, mostly dependency install, and it's slowing down every pipeline run across the team. How do you actually fix it? TRICKY REAL-TIME

Layer caching is the first lever, and the most common mistake is ordering — if you COPY the entire application source before installing dependencies, any code change invalidates the cache for the dependency-install layer too, forcing a full reinstall every build. Fix: COPY only the dependency manifest (package.json/requirements.txt/pom.xml) first, run the install as its own layer, then COPY the rest of the source afterward — now code changes don't bust the dependency cache, and only a manifest change does. Second lever: multi-stage builds — do the heavy build/compile work in a builder stage with all the build tooling, then COPY just the final artifacts into a minimal runtime stage, which also shrinks the final image significantly. Third, for CI specifically: make sure the CI system is actually reusing Docker layer cache between runs (registry-based cache with --cache-from, or a CI-native layer cache) rather than starting from a cold Docker daemon every single build, which is a very common and very expensive default in ephemeral CI runners.

```
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-slim
COPY --from=builder /app/dist /app
CMD ["node", "/app/index.js"]
```

Q2. COPY versus ADD — is there ever a real reason to use ADD?

COPY is the default for everything — it's simple, predictable, and does exactly what it says. ADD has two extra behaviors that make it riskier than useful most of the time: it auto-extracts local tar archives, and it can fetch remote URLs directly into the image. The URL-fetch behavior specifically is a real anti-pattern — no build-time layer caching benefit, no checksum verification unless you add it yourself, and it silently couples your build to network availability and whatever that URL currently serves, which is bad for reproducibility. The one legitimate case for ADD is genuinely wanting the tar-auto-extract behavior — pulling a local .tar.gz and having it extracted into the image in one step. Outside that narrow case, I standardize on COPY across a team, and if the extract-a-tarball case comes up, I'd rather RUN tar -xzf explicitly anyway so the behavior is obvious to the next person reading the Dockerfile instead of relying on ADD's implicit magic.

Runtime Behavior

Q1. You docker exec into a running container and fix a config file live to unblock an incident. Twenty minutes later the container restarts on its own and the fix is gone. Why, and what should you actually have done? TRICKY REAL-TIME

Anything changed inside a running container's writable layer is ephemeral — it lives only in that container instance's layer, not the image, and the moment the container restarts (crash, orchestrator reschedule, node reboot, a Kubernetes liveness probe killing it), you get a fresh container from the original image, and the exec'd change is gone, not because Docker did anything wrong but because that's exactly what 'stateless, restart-from-image' means. What I should have done: treat the exec fix as strictly a diagnostic/verification step — confirm the fix actually resolves the issue live — then immediately encode that same fix into the image (Dockerfile change) or into whatever config-injection mechanism is actually in use (ConfigMap in Kubernetes, an Ansible-templated config baked at deploy time), rebuild, and redeploy through the normal pipeline. docker exec is for diagnosis and emergency mitigation buying time, never for a durable fix — that's a discipline I'd want the whole team holding to, because the failure mode (silent revert on next restart, usually at the worst time) is exactly what just happened here.

Q2. What's the actual difference between ENTRYPOINT and CMD, and what happens when you use both?

CMD alone sets the default command, fully overridable by anything passed after the image name at docker run. ENTRYPOINT alone sets the command that always runs, and anything passed at docker run gets appended as arguments to it rather than replacing it. Used together — ENTRYPOINT as the fixed executable, CMD as its default arguments — you get a container that behaves like a proper CLI tool: ENTRYPOINT ["myapp"] with CMD ["--config", "/etc/default.conf"] means docker run myimage always runs myapp, using the default config unless you override just the arguments at runtime with docker run myimage --config /etc/other.conf. The gotcha people hit: shell form (ENTRYPOINT myapp) versus exec form (ENTRYPOINT ["myapp"]) behave differently around signal handling — shell form runs the command as a child of a shell process, so SIGTERM from docker stop goes to the shell, not your app, meaning graceful shutdown handlers in your app never fire and Docker eventually SIGKILLs it after the timeout. I always use exec form for exactly this reason.

Q3. Real gotcha: you're debugging a slow-leak memory issue in a container. You docker stop it, docker rm it, but the volume it wrote to still has all the data — and separately, a colleague asks if a docker rm -f on a container that had an active exec session loses data. Walk through both.

TRICKY REAL-TIME

First part: yes, named volumes and bind mounts are explicitly designed to outlive the container — that's the entire point of using a volume instead of the container's writable layer, so stopping and removing the container is safe for anything that genuinely needs to persist, and re-running a new container against the same volume picks up right where the old one left off. The gotcha to watch for is anonymous volumes (declared in the image but not explicitly named) — those can get orphaned and accumulate as dangling volumes over time if containers using them are removed without -v, which is worth a periodic docker volume prune in CI hosts that build a lot of images. Second part: docker rm -f sends SIGKILL to the main process immediately, no graceful shutdown — if there's an active exec session with an in-flight operation (a write mid-flush, a transaction not yet committed), that work is lost exactly like any ungraceful process kill would lose it; it has nothing to do with the exec session specifically, it's the same risk as SIGKILLing any process mid-write. The safe sequence for anything with in-flight state is docker stop (SIGTERM, grace period, then SIGKILL only if it doesn't exit) rather than jumping straight to -f.

Kubernetes & OpenShift

OpenShift Specifics

Q1. This JD lists both Kubernetes and OpenShift — what actually differs day-to-day, beyond 'OpenShift is Kubernetes with extras'?

OpenShift is a Kubernetes distribution, so the core object model (Pods, Deployments, Services) is the same, but a few differences genuinely change daily workflow. Security defaults: OpenShift's Security Context Constraints (SCCs) are stricter by default than raw Kubernetes' PodSecurityPolicy/PodSecurity-admission — OpenShift refuses to run containers as root by default (assigns a random high UID) unless you explicitly request a specific SCC, which means Dockerfiles and Helm charts written assuming root or a fixed UID often need adjustment to run cleanly on OpenShift. Build integration: OpenShift has BuildConfig and native source-to-image (S2I) builds, letting you go from source to a running deployment without hand-writing a Dockerfile in some workflows — different from vanilla K8s which assumes you bring your own pre-built image. Routing: OpenShift Routes versus vanilla K8s Ingress — similar purpose, different object and slightly different feature set (OpenShift Routes have built-in TLS termination options and traffic splitting that Ingress needs an extra controller/annotations to match). And oc as the CLI, which wraps kubectl but adds OpenShift-specific verbs (oc new-app, oc rollout, project-scoping by default in a way plain K8s namespaces don't enforce as strongly).

Q2. A container image that runs fine on your local Docker and on a vanilla K8s cluster fails to start on OpenShift with a permissions error. What's actually going on? TRICKY REAL-TIME

Almost certainly OpenShift's default SCC (restricted, or restricted-v2 on newer versions) assigning the container a random, non-root, non-predictable UID rather than whatever UID your image's Dockerfile specifies or defaults to — if your application writes to a directory owned by a specific UID (say, a hardcoded UID 1000 baked into the image) and OpenShift actually runs it as some arbitrary UID like 1000650000, that write fails with a permissions error even though the exact same image ran fine as UID 1000 elsewhere. The real fix isn't to just grant the image a privileged SCC to make the error go away — that's a security regression and usually gets flagged in review anyway. Proper fix: make the image UID-agnostic — set directory permissions with group ownership (GID 0, since OpenShift's random UID is still in the root group by convention) and 'g=u' style permissions so any UID in that group can read/write, which is the standard S2I-compatible image pattern. I'd only escalate to requesting an SCC exception (like anyuid) as a last resort for something that genuinely can't be made UID-agnostic, and even then, that request goes through whoever owns OpenShift security policy, not a self-service workaround.

Troubleshooting

Q1. A pod is stuck in CrashLoopBackOff, but kubectl logs shows nothing useful — just an empty or truncated log right before it dies. What's your actual debugging sequence? TRICKY REAL-TIME

First, kubectl logs <pod> --previous — the crashed container's logs, not the new restart attempt's (mostly-empty) logs, since by the time you look, Kubernetes has often already spun up attempt number 4 or 5 and your --previous-less kubectl logs is showing you the newest attempt's few seconds of output, not the actual crash. Second, kubectl describe pod for the Events section — OOMKilled shows up there distinctly from a genuine application crash, and that single word changes the entire debugging direction (resource limits problem, not code problem). Third, if it's crashing too fast to get meaningful logs even with --previous, I'd temporarily override the container's command/args to something that keeps it alive (sleep infinity) via a debug deployment or kubectl debug, then exec in and manually run the actual startup command to see the real failure interactively, since sometimes the fast-crash itself is masking a stdout/stderr buffering issue where output genuinely never gets flushed before the process dies. Fourth, check readiness/liveness probe config — a probe with too aggressive a timeout can be killing an app that's just slow to start, which looks identical to a real crash from the outside.

Q2. Explain the difference between a Deployment's rolling update getting stuck versus a pod being Running but not Ready, and how you'd tell which is happening.

A rolling update stuck usually shows as kubectl rollout status hanging, and kubectl get rs showing the new ReplicaSet not reaching desired replica count while the old ReplicaSet still has pods — the rollout is blocked by maxUnavailable/maxSurge constraints waiting on new pods to become Ready before it'll proceed further, so it's genuinely stuck on readiness, not a separate issue. Running-but-not-Ready is the actual root state underneath that — the pod's container process started successfully (Running) but its readiness probe is failing, meaning Kubernetes won't route Service traffic to it and, during a rollout, won't consider the update progressed. I'd check kubectl describe pod for the readiness probe's actual failure (wrong path, wrong port, app genuinely not ready to serve — like still warming a cache or waiting on a DB connection that isn't up yet), and kubectl get endpoints for that Service to confirm the pod's IP is genuinely absent from the endpoint list, which is the concrete, checkable symptom of not-Ready translating into 'no traffic.'

Resource Management

Q1. You set requests and limits on a pod, but under load it still gets OOMKilled well before hitting what you thought was the memory limit. What's the actual mismatch? TRICKY REAL-TIME

Most common real cause: the application's own memory settings don't know about the container's cgroup limit at all — a JVM, for example, without -XX:MaxRAMPercentage or a modern JVM version with proper container awareness, may calculate its heap sizing off the node's total physical memory rather than the container's limit, and happily grow well past what the container is actually allowed, at which point the kernel's OOM killer (via the cgroup, reported by Kubernetes as OOMKilled) terminates it — the app never got a graceful 'you're low on memory' signal, it just got killed. Second cause: the limit accounts for the main process but not for genuine overhead — sidecar containers in the same pod each need their own requests/limits, and if you only sized the main container, a sidecar (a logging agent, a service mesh proxy) consuming memory you didn't budget for pushes the pod over. My fix: confirm the runtime is actually container-memory-aware (modern JVMs are, by default, since JDK 10+, but older JVMs or explicit -Xmx flags that ignore cgroup limits need explicit correction), set requests close to steady-state usage and limits with real headroom based on observed peak, not a guess, and size every container in the pod, not just the obvious one.

Q2. What happens if you set a memory limit but no memory request, or vice versa — why does the combination matter?

If you set only a limit with no request, Kubernetes defaults the request to equal the limit — which actually removes your scheduling flexibility, since the scheduler now reserves the full limit's worth of memory on a node even if the pod typically uses far less, wasting bin-packing efficiency across the cluster. If you set only a request with no limit, the pod can consume unbounded memory on that node (in Kubernetes versions/configs without a default LimitRange), which is dangerous in a shared cluster — one misbehaving pod without a limit can pressure or evict well-behaved neighbors sharing that node. The combination that actually gives you both good scheduling and good isolation: request set to typical/steady-state usage (what the scheduler uses for bin-packing decisions) and limit set higher, with enough headroom for legitimate spikes but still bounded — and for CPU specifically, I'd generally avoid setting a hard CPU limit at all unless there's a real multi-tenant noisy-neighbor concern, since CPU throttling under a limit can cause latency spikes that are often worse than just letting it burst, whereas memory always needs a hard limit because there's no equivalent of graceful throttling for memory — it's OOM-kill or nothing.

Networking

Q1. A Service has endpoints and pods are Ready, DNS resolves the Service name correctly, but requests to it intermittently time out. Where do you look next? TRICKY REAL-TIME

Since basic connectivity/DNS/readiness all check out, I'd suspect either a NetworkPolicy silently dropping some traffic, or a load-balancing/session-affinity mismatch causing intermittent (not total) failure specifically. Concretely: check for a NetworkPolicy scoped to that namespace or those pod labels that allows most traffic but not all — a policy written for one caller's IP range/label selector that doesn't cover a second legitimate caller shows up exactly as 'intermittent,' since only requests from the disallowed source fail while everything else works. Second: kube-proxy mode and iptables/IPVS rule staleness on a specific node — if the Service's endpoint list changed recently (a rollout, a pod eviction) and a node's kube-proxy hasn't fully synced its rules, traffic routed through that specific node can hit a stale/dead endpoint intermittently until it catches up; I'd check whether the timeouts correlate with a specific node rather than being uniformly distributed. Third: if it's a long-lived connection setup (gRPC, keep-alive HTTP) hitting a Service, versus fresh connections each time — some clients cache a resolved endpoint and don't rebalance on pod churn the way a fresh connection through the Service's virtual IP would, which shows up as intermittent timeouts specifically after a rollout. I'd correlate the timeout pattern against recent rollouts, specific source pods/nodes, and NetworkPolicy changes before assuming it's a generic 'flaky network' issue.

Scripting

Shell

Q1. A shell automation script that's run hundreds of times without issue suddenly deletes more files than expected during a cleanup run, on one specific host. What class of bug do you suspect first, and how do you prevent recurrence? TRICKY REAL-TIME

First suspect: an unquoted or unset variable expanding to something unintended — a classic case is a cleanup script like rm -rf $BASE_DIR/* where $BASE_DIR was somehow empty on that one host (a missing environment variable, a failed earlier command that should have set it but didn't error loudly), collapsing the command to rm -rf /* effectively, or at least rm -rf /* relative to wherever the script happened to be running from. This is exactly why set -u (treat unset variables as an error, not silent-empty) should be standard in any script doing destructive operations, alongside set -e and pipefail. Second suspect: a glob expanding differently than expected because of a working-directory assumption that held on every other host but not this one — a relative path in a script that assumed it always runs from a specific directory. My concrete prevention: set -euo pipefail as a non-negotiable header on every destructive script, always quote variable expansions ("$VAR" not $VAR), validate that any variable used in a destructive command is both set and non-empty with an explicit check before the destructive command runs, and for anything genuinely destructive, a dry-run mode (echo the rm commands instead of running them) that gets exercised before the real flag is ever set.

```
set -euo pipefail

if [[ -z "${BASE_DIR:-}" ]]; then
  echo "ERROR: BASE_DIR is unset, refusing to proceed" >&2
  exit 1
fi

rm -rf "${BASE_DIR:?}"/*
```

Q2. What's your go-to pattern for a shell script that needs to run reliably as part of a deployment automation pipeline — what does 'production-grade shell' look like to you?

set -euo pipefail at the top, always. Explicit error trapping with trap for cleanup on exit/failure, so a script that dies halfway doesn't leave a lock file or a half-copied artifact behind. Logging that's actually useful under pressure — timestamped, to both stdout (for pipeline console capture) and a persistent log file if it's long-running. Idempotency where possible — check state before acting, same discipline as Ansible modules, rather than assuming a clean starting state every run. Input validation on anything parameterized rather than trusting arguments blindly, since a pipeline-triggered script often receives parameters from upstream that could be malformed. And for anything beyond fairly simple glue logic, I'll actually reach for Python instead of pushing shell past its comfortable complexity ceiling — shell is great for orchestration and simple file/process operations, but complex logic, real error handling, and anything needing structured data (JSON parsing, API calls) gets unreliable and unreadable in shell past a certain point.

Python

Q1. You wrote a Python automation script that calls an internal API to trigger deployments across dozens of on-prem hosts. It works great until one host's API call hangs, and now the whole script is stuck with no timeout, no way to know which host it's on. TRICKY REAL-TIME

Two separate real problems here worth fixing independently: no timeout on the request itself, and no visibility into progress. requests without an explicit timeout= will hang indefinitely if the remote side never responds — that's a very common real bug, since Python's requests library has no default timeout, which surprises people coming from other ecosystems. Fix: always set a timeout, and wrap the call so a timeout on one host is caught and logged rather than propagating as an unhandled hang. For visibility, I'd add structured logging per-host with the current host name and step printed before each blocking call, so if it does hang again, you know exactly which host and which step from the last log line rather than staring at a frozen terminal. For genuine resilience across many hosts I'd also move from sequential to a thread/process pool with per-host timeouts and a bounded concurrency limit, so one hung host doesn't block the other 39 that are perfectly fine — collect results and failures into a report at the end rather than the whole run living or dying on one bad host.

```
import requests

try:
    resp = requests.post(url, json=payload, timeout=(5, 30))  # connect, read
    resp.raise_for_status()
except requests.exceptions.Timeout:
    log.error(f"{host}: deploy call timed out")
    failures.append(host)
except requests.exceptions.RequestException as e:
    log.error(f"{host}: {e}")
    failures.append(host)
```

Q2. When do you reach for Python over Ansible for an automation task, given they overlap?

Ansible when the task is fundamentally 'converge these hosts to a desired state' — configuration management, package installs, service management — where idempotent modules and inventory-driven execution are the natural fit and writing it in raw Python would just be reinventing what Ansible already does well. Python when the task is genuinely programmatic — calling internal REST APIs, parsing and transforming structured data (JSON/XML reports, log analysis), building a custom tool with real control flow and error handling that would be awkward to express as a playbook, or gluing together multiple systems (querying a CMDB, then deciding what to do based on the result, then triggering Ansible itself). A common real pattern: Python as the orchestration layer that decides what needs to happen and calls Ansible (via ansible-runner or subprocess) to actually make hosts converge — Python for the decision logic and API integration, Ansible for the actual host-state work, rather than picking one tool to do both jobs.

Groovy

Q1. What's genuinely different about writing Groovy for Jenkins pipelines versus writing 'normal' Groovy or Java-adjacent code?

The big one is Jenkins' CPS (Continuation-Passing-Style) transformation, which rewrites your pipeline Groovy under the hood to support pausing and resuming execution (needed for things like input steps waiting on human approval, or a Jenkins restart mid-pipeline not losing state). That transformation doesn't play well with everything — certain Java standard library operations, some closures, and non-serializable objects can behave unexpectedly or throw NotSerializableException inside CPS-transformed code in ways they never would in plain Groovy or Java. The practical fix is the @NonCPS annotation on methods that do pure computation and don't need to survive a pause/resume, which exempts them from the transformation and lets you write normal, fast Groovy/Java-style code inside them — I use @NonCPS routinely for things like parsing a JSON string or doing string manipulation inside a shared library, where CPS transformation offers no benefit and only adds overhead and occasional serialization headaches.

Observability

Prometheus

Q1. Prometheus alerting is completely silent during a real outage — no pages fired — even though the dashboard clearly shows the metric crossed the alert threshold twenty minutes ago. How do you debug alerting itself? TRICKY REAL-TIME

I'd separate this into three distinct failure points, because 'alerting is silent' could be broken at any of them: the alert rule itself never fired, it fired but Alertmanager never received it, or Alertmanager received it but routing/silencing suppressed the notification. Check order: first, Prometheus's own alert state via the /alerts endpoint or UI — if the rule shows 'pending' or never transitioned to 'firing,' check the for: duration on the rule, since a metric crossing threshold briefly and dropping back under the for: window never actually fires, which is by design but easy to misconfigure too aggressively. If it did fire, check Alertmanager received it — Prometheus's alerting config needs to actually point at the right Alertmanager endpoint, and a misconfiguration there means alerts fire in Prometheus but silently vanish, never reaching Alertmanager at all. If Alertmanager received it, check for an active silence (someone silenced this exact alert during a maintenance window that never got cleared, which is an extremely common real cause of 'alerting was silent during a real incident') or a routing tree misconfiguration sending it to a receiver nobody's actually watching. I'd genuinely check silences first in practice — a forgotten, un-expired maintenance silence is disproportionately often the actual root cause when 'nothing went wrong with the pipeline, alerting was just silenced.'

Q2. Explain the difference between a counter, gauge, histogram, and summary in Prometheus, with when you'd actually use each.

Counter — monotonically increasing, only resets on restart; use for things that only go up, like total requests served or total errors, and always query it with rate() or increase() rather than the raw value, since the raw cumulative number alone isn't useful. Gauge — can go up or down; use for point-in-time values like current memory usage, active connections, queue depth. Histogram — buckets observations (like request latency) into configurable ranges and exposes counts per bucket plus a total sum and count, letting you calculate percentiles server-side at query time via histogram_quantile() — the buckets are pre-defined at instrumentation time, so choosing sensible bucket boundaries for your actual latency distribution matters, or your percentile calculations get inaccurate. Summary — also for distributions like latency, but calculates configurable quantiles (p50, p99) client-side at scrape time rather than letting you compute them flexibly later — cheaper to query but you can't aggregate summary quantiles across multiple instances meaningfully (averaging p99s from 10 pods isn't the true p99 across the fleet), which is exactly why histogram is generally preferred for anything you need to aggregate across a distributed system, and summary is mostly reserved for single-instance-scoped metrics where cross-instance aggregation was never needed.

ELK Stack

Q1. Elasticsearch cluster health goes yellow, then intermittently red, right when log ingestion volume spikes during an incident — the worst possible time. What's actually happening and what's the fix? TRICKY REAL-TIME

Yellow means some replica shards are unassigned (still tolerable, degraded), red means primary shards are unassigned (real data unavailability for those indices) — and this happening exactly during a log-volume spike is a very common, very real failure mode: heavy indexing pressure exhausts either disk space (hitting the disk watermark thresholds that Elasticsearch uses to stop allocating/writing to a node) or JVM heap (indexing is memory-intensive, and heap pressure triggers circuit breakers or even node instability), and a struggling/overloaded node can drop out of the cluster, taking its primary shards' availability with it until they're reassigned or the node rejoins. Immediate triage: check disk usage against the watermark thresholds (cluster.routing.allocation.disk.watermark) first, since that's the single most common real cause — Elasticsearch is being conservative and correct by refusing writes/allocation near a full disk, it's not misbehaving. Check node/JVM heap pressure via the cluster stats API. Mitigation during the incident itself: if it's genuinely disk pressure, free space or add capacity fast, and consider throttling non-critical index ingestion temporarily to protect cluster stability over completeness of every log line. Root-cause fix afterward: proper index lifecycle management (ILM) so old indices roll off automatically before disk pressure becomes a recurring incident-time surprise, and capacity planning that accounts for burst ingestion during exactly the scenario that's most likely to also be a real incident — which is an unpleasant but real correlation, since incidents themselves often generate more log volume.

Q2. Walk through the actual role each component plays in the ELK/Elastic stack, and where Logstash fits versus just shipping logs directly.

Elasticsearch — the storage and search/indexing engine, where logs actually live as documents in an inverted index for fast full-text and structured search. Logstash — the ingestion and transformation pipeline: parses raw, often messy log lines (grok patterns for unstructured text, or structured JSON parsing) into structured fields, enriches (adding metadata, geo-IP lookups), and routes to Elasticsearch (or elsewhere). Kibana — the visualization and query UI on top of Elasticsearch. Where it gets a real design decision: Logstash is genuinely heavyweight (JVM-based, resource-hungry) for pure shipping, so a lot of real-world architectures use Filebeat (lightweight, Go-based) to tail and ship logs, and either send straight to Elasticsearch for already-structured logs (application logs already emitting JSON), or through Logstash specifically when real parsing/transformation of unstructured text is needed (legacy app logs with no structured format, needing grok patterns to extract fields). I've built pipelines both ways — Filebeat direct to Elasticsearch for cleanly-structured app logs, Filebeat to Logstash to Elasticsearch when there's genuine unstructured legacy log format to parse — picking based on whether transformation is actually needed, not defaulting to the heaviest tool by habit.

OpenTelemetry

Q1. What problem does OpenTelemetry actually solve that Prometheus + ELK alone don't, and how does it fit alongside them rather than replace them?

OpenTelemetry standardizes instrumentation and, critically, adds distributed tracing as a first-class signal alongside metrics and logs — the three pillars of observability together (traces, metrics, logs), correlated. Prometheus gives you metrics, ELK gives you logs, but neither natively answers 'this specific request was slow — which of the 6 microservices it touched actually caused the delay, and where exactly in that service.' Distributed tracing, via OpenTelemetry SDKs instrumenting each service and propagating a trace context (trace ID, span ID) across service boundaries, answers exactly that — you get a waterfall view of one request's full path across every service it touched, with per-span timing. OpenTelemetry doesn't replace Prometheus or ELK — it's a vendor-neutral instrumentation and collection layer that can export metrics to Prometheus, traces to a tracing backend (Jaeger, Tempo, or a commercial APM), and it increasingly can correlate logs with trace IDs too, so you can jump from a slow trace directly to the exact log lines from that specific request. The real value for a platform team is the vendor-neutral part — instrument once with OTel SDKs/the Collector, and you're not locked into re-instrumenting every service if the backend (Jaeger versus a commercial APM versus something else) changes later.

Q2. You've instrumented a service with OpenTelemetry and traces show up fine for most requests, but traces for calls going through an async message queue (Kafka/RabbitMQ) show up as broken, disconnected spans instead of one continuous trace. Why? TRICKY REAL-TIME

Trace context propagation is the mechanism that stitches spans across service boundaries into one trace, and it normally rides along with the request — HTTP headers, gRPC metadata. The moment a request's flow goes through an async queue, there's no synchronous request/response carrying that context automatically — the trace context (trace ID, parent span ID) has to be explicitly injected into the message when it's published, and explicitly extracted when it's consumed, or the consumer side starts a brand-new, disconnected trace with no idea it was ever part of the original request's trace. This is a very real, very common gap when teams instrument HTTP/gRPC paths (where OTel's auto-instrumentation often handles propagation transparently) but don't realize async messaging needs manual context propagation — injecting trace headers into the message's own headers/metadata field on publish, and extracting them on the consumer side to continue the same trace rather than start a new root span. Fix is instrumentation-level, not infrastructure-level: use the OTel messaging semantic conventions and make sure both the publisher and consumer sides of the queue are explicitly propagating and continuing context, not just auto-instrumented for whatever synchronous framework they're also using.

Artifactory & Nexus

Repository Management

Q1. What's the actual role of Artifactory/Nexus in a pipeline beyond 'a place to store build artifacts'?

Single source of truth for anything a build depends on or produces, which is the real value: proxy repositories cache external dependencies (Maven Central, npm registry, Docker Hub) so builds don't depend on public internet availability and aren't exposed to a public registry silently changing/removing a package version out from under you; hosted repositories store your own built artifacts, versioned and immutable once published, which is what makes 'redeploy exactly what was tested' actually possible rather than aspirational; and for enterprise/regulated on-prem environments specifically, it's the control point for governance — vulnerability scanning integration (Artifactory Xray), license compliance checks, retention policies, and access control over who can publish/promote artifacts toward production. I'd also highlight repository promotion — moving an artifact from a snapshot/dev repo to a release repo as an explicit, audited step once it passes gates, rather than every environment just pulling from one undifferentiated bucket of artifacts.

Q2. A production deploy pulls what should be a known-good, previously-tested artifact version from Nexus, but the actual bytes deployed are subtly different from what passed testing three days ago. How is that even possible if versions are supposed to be immutable? TRICKY REAL-TIME

A few real ways this happens, and none of them mean Nexus itself is broken: someone republished the same version number over the old artifact — release repositories should be configured to reject redeploys/overwrites of an existing version (Nexus and Artifactory both support this as a repository setting), and if that setting isn't enforced, a well-meaning 'I just need to patch this one thing' redeploy under the same version number silently replaces what was actually tested, which is a governance gap, not a tooling limitation. Second: it wasn't actually a release-repo artifact at all — if the pipeline pulled from a snapshot repo (mutable by design) rather than a promoted release artifact, 'same version string' never guaranteed immutability in the first place, and that's a pipeline configuration bug, pulling from the wrong repo type for a prod deploy. Third, less common but real: a proxy/cache layer serving a stale or corrupted cached copy that never got properly invalidated. My actual fix: enforce release-repo immutability (reject overwrite) as a hard policy setting, not a convention; verify the pipeline's prod deploy step pulls specifically from the release repo, never snapshot; and add artifact checksum verification (Nexus/Artifactory both expose SHA checksums) as an explicit pipeline step comparing what's deployed against what was tested, so this class of drift is caught automatically rather than discovered after the fact.

Cloud AWS/Azure

Hybrid & CI/CD Integration

Q1. This role is on-prem-heavy but lists AWS/Azure and Azure CI/CD pipelines — what's the realistic hybrid pattern you'd expect, and how do you approach it?

Most enterprises in a genuine hybrid position aren't fully migrating — they're running CI/CD orchestration and some services in the cloud (Azure DevOps as the pipeline engine, for instance) while workloads still run on-prem, or they're in a partial migration where some services have moved and others haven't, with both needing to be managed consistently. My approach: treat the pipeline tooling (Azure DevOps, Jenkins) as the control plane regardless of where the workload runs — the same pipeline concept (build, scan, package, deploy) should work whether the deploy target is an on-prem VM via Ansible, an on-prem Kubernetes/OpenShift cluster, or an Azure/AWS resource, just with a different deploy stage plugged in. I'd also expect network connectivity between on-prem and cloud (VPN or ExpressRoute/Direct Connect) to be a real constraint worth understanding early — a pipeline agent that needs to reach both an on-prem artifact repo and an Azure resource has real latency and firewall/routing dependencies that don't exist in a pure-cloud setup, and I'd want to know that topology before assuming a design will just work.

Q2. An Azure DevOps pipeline needs to deploy to an on-prem server behind the corporate firewall, with no inbound internet access. How do you actually architect that connection? TRICKY REAL-TIME

Azure DevOps self-hosted agents are the mechanism — instead of using Microsoft-hosted build agents (which live outside your network and can't reach an on-prem server with no inbound access), you install a self-hosted agent on a machine inside the on-prem network (or in a DMZ with appropriate access), and that agent polls Azure DevOps over outbound HTTPS to pick up jobs — no inbound firewall hole needed at all, since the agent initiates the connection outward, not the other way around. The agent then executes the deploy step locally within the network, with direct access to the on-prem target it needs to reach (SSH to run Ansible, direct network access to a Kubernetes/OpenShift API server, whatever the actual deploy mechanism is). I'd think about agent pool sizing and placement carefully in a UK/Poland multi-site setup specifically — agents ideally live close to (or within) the network segment they're deploying into, both for latency and because it avoids needing broad cross-site firewall rules just for pipeline traffic.

Core Services & IAM

Q1. With both AWS and Azure certifications/exposure expected here, what's your mental model for translating IAM concepts between the two when you're context-switching?

Core concept maps fairly cleanly even though the mechanics differ: AWS IAM policies (JSON, attached to users/roles/groups, evaluated with explicit-deny-wins logic) map to Azure RBAC role assignments (built-in or custom roles, assigned at a scope — subscription/resource group/resource — rather than attached the AWS way). The genuinely different piece is Azure's scope hierarchy — management group → subscription → resource group → resource, with role assignments inheriting downward — versus AWS's flatter account/OU structure under AWS Organizations with SCPs layered on top; they're solving a similar governance problem but the actual permission-inheritance model isn't a 1:1 translation, and I've seen people assume Azure resource groups behave like AWS accounts when they're really more like a folder-level grouping within one account/subscription. For service identity specifically — AWS IAM roles for EC2/Lambda versus Azure Managed Identities — both solve 'let this compute resource authenticate without a hardcoded credential,' and I treat that equivalence as solid; it's the org-structure/scope layer where I'm careful not to over-map one onto the other.

Release Management

Coordination & Process

Q1. How do you actually coordinate a production release across teams in different countries — specifically UK and Poland, like this JD — without it turning into a communication mess?

A written release plan that's the single source of truth, not tribal knowledge scattered across Slack/Teams threads — what's shipping, who owns each component, the deploy sequence, rollback steps, and explicit go/no-go criteria at each stage, shared and reviewed by both teams ahead of time, not discovered live during the release window. Time zone overlap matters concretely — UK and Poland are close (typically only an hour apart, CET vs GMT/BST), which is a real advantage over a US/Poland split, but I'd still pin an explicit release window in UTC that both teams confirm against their local time, since 'let's do it at 2pm' silently assumes a shared time zone that isn't actually shared. Clear ownership per stage — who's the go/no-go decision-maker if something looks marginal mid-release, since ambiguity there under pressure is where releases stall. And a single communication channel for the release itself (a dedicated bridge/war-room channel), so status updates aren't fragmented across parallel threads that different people are watching.

Q2. Mid-release, the UK team says a dependent service is ready to cut over, but the Poland team — who actually own that service's pipeline — say their last deploy hasn't finished validating. Release window is closing in 30 minutes. What do you actually do? TRICKY REAL-TIME

I don't let the clock override the actual technical state — a closing window is a scheduling problem, going live on unvalidated state is a production-risk problem, and those aren't the same magnitude of consequence, so I'd default to trusting the team who owns the pipeline and actually has eyes on validation status over the team who wants to proceed. Concretely: get both teams on the same call immediately rather than resolving through async messages, have the Poland team state precisely what's still validating and their honest ETA, and make the go/no-go call based on that, not on window pressure — if validation genuinely can't complete in 30 minutes, the right call is usually to slip the window, not cut over on hope. I'd also separate 'this specific dependent service isn't ready' from 'the whole release is blocked' — if other components are genuinely independent and validated, a partial release or rescheduling just the blocked piece might be viable, but that's a call I'd make explicitly and communicate clearly, not something that happens by default because one piece got quietly skipped under time pressure. After the fact, the real process fix is making sure validation status is visible to both teams in real time during the window, not something the UK team has to ask about and hope is current.

Q3. What does a good rollback plan actually contain, beyond 'we can roll back if needed'?

Specificity is the whole point — 'we can roll back' isn't a plan, it's an aspiration. A real rollback plan states: the exact previous artifact version/image tag to redeploy (already sitting in Nexus/Artifactory/the registry, not something you'd have to rebuild under pressure), the exact rollback command or pipeline job to trigger, explicit database migration considerations if the release included schema changes (can the old code version even run against the new schema, or does rollback also need a reverse migration — this is the part people forget most often and the part that turns a clean rollback into a genuine incident), the specific health checks/metrics that would confirm rollback succeeded, and a clear decision owner and threshold for triggering it — what specific signal, seen by whom, triggers the rollback decision, decided in advance rather than debated live during an incident. I always ask the database-migration question explicitly in release planning, because a rollback plan that looks solid for application code often silently assumes the schema is backward-compatible without anyone having actually verified that.

Enterprise Infra

Networking

Q1. Real-time scenario: an application team reports intermittent connection timeouts to a database, only under load, only for about 30 seconds at a time, then it clears. Where do you look, systematically? TRICKY REAL-TIME

Layer by layer rather than guessing: application-level first — connection pool exhaustion is the single most common cause of exactly this symptom (works fine at low load, times out under load, self-clears once load drops and connections free up), so I'd check the app's connection pool size against actual concurrent demand under load before touching networking at all. If pooling looks fine, move to the network layer — check for SNAT port exhaustion if this traffic passes through a NAT gateway or load balancer doing source NAT, since a burst of new connections under load can exhaust the available ephemeral port range on the NAT device, causing new connections to fail until existing ones age out — this shows the exact 'intermittent, load-correlated, self-clearing' signature described here. Also check the database's own max_connections limit and connection queue behavior under load, and whether there's a firewall or security appliance doing connection tracking with a table that's filling up under burst load. My actual triage order: reproduce or correlate the timeout window against connection pool metrics first (cheapest to check, most common cause), then NAT/firewall connection-table metrics, then database-side connection limits, rather than jumping straight to a packet capture, which I'd only pull if the above don't explain it.

Q2. Explain load balancing algorithms you've actually worked with — round robin, least connections, IP hash — and when each is the wrong choice.

Round robin — simplest, cycles requests evenly across backends; wrong choice when backend requests have genuinely uneven cost (some requests are cheap, some expensive) since even distribution of request count doesn't mean even distribution of actual load, or when backends have meaningfully different capacity, since round robin doesn't account for that at all. Least connections — routes to whichever backend currently has the fewest active connections, generally better for uneven request costs since it's reactive to actual current load rather than assuming uniformity; wrong choice for very short-lived connections where 'current connection count' barely differentiates backends before the next decision is needed anyway. IP hash — routes based on client IP, giving session affinity/stickiness without needing cookies; the real gotcha is that it can create uneven load distribution if your client IP distribution is skewed (a large NAT'd office network all hashing to the same backend), and it breaks badly if a backend goes down, since the hash ring shifting can suddenly send previously-sticky clients to a completely different backend, invalidating any local session state they depended on. In practice I default to least connections for most HTTP workloads unless there's a specific session-affinity requirement, and I'm cautious with IP hash unless I've actually checked the real client IP diversity feeding into it.

Storage & Servers

Q1. What's your general enterprise technical aptitude around storage — SAN versus NAS, and where does that distinction actually matter operationally?

SAN (Storage Area Network) presents block-level storage over a dedicated network (Fibre Channel or iSCSI) — the server sees it as a raw disk/LUN and puts its own filesystem on top, which is what you want for things needing low-latency, high-throughput block access, like database storage or VM datastores. NAS (Network Attached Storage) presents file-level storage over standard IP networking (NFS, SMB) — the filesystem already exists on the NAS side, and clients mount a shared filesystem directly, which is the natural fit for shared file access across multiple servers (shared config, shared application data, home directories). Operationally where it actually matters: performance characteristics and failure modes differ — a SAN LUN issue affects one server's block device (though could be one VM host serving many VMs, widening blast radius that way), while a NAS outage affects every client with that share mounted simultaneously, which is a genuinely different blast-radius shape worth thinking about when deciding what goes where. I'd also flag multipathing (multiple physical paths from server to SAN, for both performance and redundancy) as a real on-prem operational concern that has no cloud-storage equivalent conversation — it's exactly the kind of physical-topology awareness that on-prem work requires and cloud abstracts away entirely.

Q2. A batch job that reads/writes heavily to an NFS-mounted directory starts failing intermittently with stale file handle errors, only during high-load windows. What's happening and how do you fix it? TRICKY REAL-TIME

Stale file handle on NFS specifically means the client's cached reference to a file (its file handle) no longer matches what the server has — commonly because the file was deleted/recreated/moved on the server side while a client still held a handle to the old inode, or because of an NFS server failover/restart that changed the underlying export in a way the client's cached handle doesn't survive. Under high load specifically, this often correlates with a batch process doing heavy temp-file churn (create, write, rename, delete in rapid succession) where a second process reading the 'same' file races against the first process's rename/delete, or with NFS server-side load causing timeouts that trigger client-side retry/reconnect behavior that doesn't always cleanly resume the same handle. My fix approach: check whether the batch job's own file-handling pattern is the actual cause — rapid create/delete/rename cycles on NFS are inherently riskier than on local storage precisely because of handle caching, so redesigning to write to a local temp location and only move the final result to NFS once complete reduces the window significantly. Separately, check NFS mount options (hard vs soft mounts change retry/failure behavior significantly — hard mounts retry indefinitely and are usually the safer default for anything that can't tolerate silent data loss on a blip) and confirm the NFS server itself isn't under genuine resource pressure during those high-load windows causing timeouts in the first place.

PostgreSQL & Unix

PostgreSQL Ops

Q1. As a DevOps engineer, not a DBA, what's the realistic depth of PostgreSQL knowledge you'd bring, and where do you hand off to an actual DBA?

I'd frame this honestly rather than overclaim DBA-level depth I don't have — my PostgreSQL exposure is operational: backup/restore procedures (pg_dump/pg_basebackup, WAL archiving for point-in-time recovery), monitoring key health signals (connection count against max_connections, replication lag on standbys, long-running/blocking queries, disk usage growth), basic performance triage (checking pg_stat_activity for what's actually running when something's slow, recognizing a missing index from an obviously slow query plan), and automating routine operational tasks (scheduled backups, vacuum/analyze scheduling, failover runbooks) via Ansible. Where I'd hand off to a DBA: genuine query optimization and schema design decisions, deep replication topology design, and anything touching data integrity risk beyond a standard runbook — I know enough to run EXPLAIN ANALYZE and recognize a red flag, not enough to redesign an indexing strategy for a complex workload. That division of labor is exactly the right one for a DevOps/SRE role — operational ownership and automation, DBA-level design and tuning as its own specialized function.

Q2. Real scenario: PostgreSQL replication lag between primary and standby suddenly spikes from milliseconds to several minutes, and it's not recovering on its own. What do you check, as the DevOps engineer first on the call before a DBA joins? TRICKY REAL-TIME

I'd check the practical, operational-level signals first, since that's genuinely within scope even without deep DBA expertise: network connectivity and latency between primary and standby (a degraded network link is a very common on-prem cause and immediately actionable), disk I/O on the standby specifically — if the standby's storage is slower or under other load, WAL replay can't keep pace with WAL generation on the primary, and that gap compounds; standby CPU/load in general, since replay is genuinely resource-intensive. I'd also check for a long-running query on the standby if it's a hot-standby serving read traffic — a long transaction can block WAL replay depending on configuration (max_standby_streaming_delay), which is a very concrete, fixable cause. And I'd check disk space on both sides — if WAL is accumulating faster than it's being shipped/applied, filling disk on either end turns a lag problem into an outage. What I'd explicitly not do is start tuning replication internals or making a call on failover myself — that's exactly the DBA-handoff point, but having already gathered network/IO/query/disk-space findings means the DBA isn't starting from zero when they join, which is the actual value I'd add in that first-responder window.

Unix Administration

Q1. Give a real example of using awk/sed/grep together to solve an actual operational problem, not just define what each does.

A realistic one: parsing an Nginx access log to find the top 10 IPs generating 5xx errors in the last hour, during an incident where you need a fast answer without spinning up a full log-analysis tool. grep first to filter to just the relevant time window and status codes, awk to extract just the IP field and status code, sort and uniq -c to count occurrences per IP, sort -rn to rank by count, head to cap it. That kind of one-liner composition — each tool doing one narrow job, piped together — is exactly the Unix philosophy in practice, and it's genuinely faster than opening Kibana for a quick triage question during an active incident when you already have shell access to the box.

```
grep ' 5[0-9][0-9] ' /var/log/nginx/access.log | \
  awk -v since="$(date -d '1 hour ago' '+%d/%b/%Y:%H')" \
    '$4 >= "["since {print $1}' | \
  sort | uniq -c | sort -rn | head -10
```

Q2. A server's load average looks fine, but the application running on it is noticeably slow, and users are complaining. Walk through what you actually check beyond the obvious top/htop glance. TRICKY REAL-TIME

Load average alone is a genuinely incomplete signal — it counts processes in runnable or uninterruptible-sleep state, and a normal-looking load average can absolutely coexist with real user-facing slowness if the bottleneck isn't CPU contention at all. I'd check iostat -x for disk I/O wait — a process stuck waiting on slow disk shows low CPU usage and normal-ish load but real application-level latency, and %iowait in top/vmstat is the tell here that a quick top glance can miss if you're only looking at the load number. I'd check vmstat for memory pressure and swap activity — if the box is swapping, everything gets slow in a way that doesn't necessarily spike load average dramatically at first. I'd check network-level latency/connection state (ss -s, netstat) if the app depends on downstream services — slowness can genuinely be 'this box is fine, what it's talking to isn't.' And I'd check for a single runaway or misbehaving process (memory leak causing GC pressure in a JVM app, for instance) with a per-process view (top sorted by memory, or pidstat) rather than assuming system-wide resource exhaustion when it might be one process. The real lesson I'd want to convey: load average is a starting point, not a diagnosis, and I'd walk CPU, disk I/O, memory/swap, and network as four separate checks rather than stopping at the first metric that looks normal.

Tricky Scenarios

High-Pressure Incidents

Q1. It's Friday, 5:45pm UK time, and you're asked to push an urgent hotfix to production before the weekend. Your gut says this needs more testing. What do you actually do? TRICKY REAL-TIME

I'd say so, directly and specifically — not 'I'm not comfortable' in the abstract, but the concrete gap: 'this hasn't been through the standard integration test suite, here's the specific risk if it's wrong, here's what we'd need to actually validate it in the next 30 minutes if we're doing this today.' I'd rather have that conversation clearly than either silently comply with a risk I flagged internally but never said out loud, or unilaterally refuse without explaining why. If the business genuinely needs it today despite the risk, that's a decision the right stakeholder can make with the risk stated plainly — I'm not the sole decision-maker on business risk tolerance, but I am responsible for making sure that risk is actually understood before the decision gets made, not discovered Monday morning. Concretely I'd also push for whatever partial validation is actually achievable in the time available (a smoke test against the exact change, not a full regression) rather than treating it as all-or-nothing, and I'd want an explicit, fast rollback plan ready given the reduced confidence — that's the part I'd insist on regardless of how the go/no-go call lands, because 'we shipped fast' and 'we can undo it fast if wrong' are two different guarantees and only one of them is negotiable under pressure.

Q2. During a live incident, two senior engineers on the call are confidently proposing two different, contradictory root causes and fixes. You have a hypothesis that's different from both. How do you handle that live, without derailing the incident? TRICKY REAL-TIME

I wouldn't argue abstractly for my theory against theirs — during an active incident that just burns time. I'd state my hypothesis briefly with the specific evidence behind it, and more importantly, propose the fastest way to actually test it rather than debate it: 'if it's X, we'd expect to see Y in the logs — can someone check that while we also look at what's being proposed.' Incidents are better served by parallel, falsifiable checks than by three people debating from intuition, so my instinct is to turn disagreement into a quick, concrete test rather than a discussion. If my hypothesis turns out to be wrong once checked, I say so plainly and move on — ego about being right isn't useful mid-incident. If time is genuinely critical and we can't test every theory, I'd defer to whoever has an incident commander role (or is acting as one) to pick a direction and commit to it, rather than letting three parallel theories all get partial, unfinished investigation simultaneously, which is often slower than picking one reasonable path and following it through.

Q3. You made a change during an incident that you believed would fix it, but it actually made things worse. How do you handle that moment, live, in front of the team? TRICKY REAL-TIME

Say it immediately and plainly — 'that was my change, it made it worse, here's what I did, rolling it back now' — the worst version of this is hesitating or downplaying it while things degrade further because owning it feels uncomfortable. Speed of disclosure matters more than anything else in that moment, because every minute someone else spends investigating a cause that's actually your own change is a minute wasted. I'd execute the rollback myself immediately if I can, or hand it to whoever can do it fastest, then only after the immediate bleeding is stopped, explain what I was trying to do and why it didn't work — that explanation is useful for the post-incident review, not for the middle of the incident itself. Afterward, in the actual postmortem, I'd walk through it blamelessly and honestly — what I assumed that turned out wrong, what check I skipped that would have caught it, what I'd do differently — because that's genuinely useful process improvement, and I've found teams respect directness about a real mistake far more than a well-hedged non-admission.

Diagnostic Puzzles

Q1. A deployment pipeline has been completely unchanged for six months, no code changes to the pipeline itself, but it's been getting progressively slower over the last three weeks — no single dramatic failure, just creeping slowness. How do you approach something with no obvious trigger?

TRICKY REAL-TIME

Gradual, trigger-less degradation over weeks points toward something accumulating rather than something that broke — that framing changes where I look first. Real candidates: artifact repository (Nexus/Artifactory) growing without cleanup, so dependency resolution against an ever-larger index gets progressively slower; a build cache that's supposed to help but has grown stale/bloated and is now doing more harm than good; disk space on the CI runner/agent creeping toward capacity, with filesystem performance degrading as it fills, especially on certain filesystem types; or genuinely growing test suite / codebase size, where 'slower' is actually organic and expected, just never noticed because nobody's been tracking pipeline duration as a metric over time. My actual approach: pull historical pipeline duration data if it exists (most CI systems track this) and look at which specific stage's duration is actually growing, rather than treating 'the pipeline' as one undifferentiated black box — that narrows the search dramatically. Then correlate that specific stage against what's been growing over the same three-week window — repo size, cache size, disk usage, dependency count — rather than guessing. This is also a good argument for why pipeline stage duration should be a tracked, alerted-on metric in the first place, so 'three weeks of silent creep' gets caught at week one instead.

Q2. A specific application only fails during deployments that happen to land between 2-4am UTC, never at any other time, and the deployment steps and code are identical every time. What's your hypothesis-generation process for something this specific?

TRICKY REAL-TIME

A time-window-specific pattern this precise almost never means the deployment logic itself is at fault — identical steps succeeding at other times rules that out directly — so I'd look for what else is scheduled or different specifically in that window. Real candidates: a scheduled batch job, backup window, or maintenance task that runs in that exact window and contends for the same resources (disk I/O from a backup job, a database maintenance/vacuum window, a batch ETL job) — deployments doing anything resource-intensive (restarting services, warming caches, running migrations) colliding with another scheduled heavy process is a very plausible real cause. Second: a downstream dependency that has its own scheduled maintenance in that window — an upstream service, a database replica sync, a certificate rotation job — that the deploying application depends on and that's simply unavailable or degraded during that specific window regardless of what's deploying. Third, less likely but worth ruling out cheaply: a time-zone-related logic bug that's coincidentally more likely to trigger during that specific UTC window (a date-rollover edge case, a scheduled cache-expiry). My actual first move: check the on-prem environment's full job/cron/scheduler inventory for anything else running 2-4am UTC before investigating the deploying application's own code at all — the specificity of the time window is the single biggest clue and it's cheap to check against a schedule inventory before going deeper.

Communication & Leadership

Cross-Team Working

Q1. What actually makes cross-country stakeholder collaboration (UK and Poland here) work well versus poorly, from your experience?

Explicit, written communication as the default, not an afterthought — decisions, action items, and status need to live somewhere both teams can reference asynchronously, not just in a meeting one side attended and the other heard about secondhand. Overlap-hour discipline — UK and Poland have a comfortable working-hours overlap (roughly an hour's time difference), which is genuinely favorable compared to many distributed setups, but I'd still be deliberate about scheduling anything requiring live discussion inside that overlap rather than assuming it'll just work out. Clear, unambiguous ownership per component/service, documented, so 'who do I even ask about this' never becomes its own delay. And genuinely treating both sides as equal technical partners rather than one team being the 'main' team and the other being support — that dynamic, even unintentional, tends to create information asymmetry where the 'support' side finds out about decisions after the fact rather than being part of making them, which erodes trust and slows everything down over time.

Q2. You're implementing a change that the Poland-based team will need to operate day-to-day after you hand it off, but you're under deadline pressure and tempted to just ship it with minimal documentation and explain it 'later.' What's the actual cost of that shortcut, and what would you do instead? TRICKY REAL-TIME

The shortcut doesn't actually save time, it moves the cost — 'explain it later' usually means multiple ad-hoc explanations repeated to different people as questions come up piecemeal, interrupt-driven, at the worst possible moments (usually when something's already gone wrong and they're troubleshooting blind), which costs more total time than writing it once, upfront, clearly. What I'd actually do under real deadline pressure: write a minimal but genuinely sufficient runbook — what this does, how to check it's healthy, the two or three most likely failure modes and what to do about each — rather than either full comprehensive documentation (too slow under pressure) or nothing (creates the exact problem above). I'd also do a short live walkthrough with the Poland team specifically, even 15 minutes, because a live walkthrough surfaces the questions they'll actually have far faster than them discovering gaps in written docs on their own later — and I'd rather find out what's unclear now, from a calm conversation, than at 2am when they're the ones on call for it.

Team & Ownership

Q1. As a senior engineer expected to operate with real autonomy across geographically distributed stakeholders, how do you decide what to escalate versus what to just handle yourself?

I escalate when a decision genuinely crosses into risk or scope beyond what I have context or authority for — real business risk tradeoffs (accepting a security finding, shipping under uncertainty, a cost decision with real budget impact), anything touching a commitment made to a stakeholder I wasn't part of the original conversation with, or a decision where getting it wrong is expensive and hard to reverse. I handle it myself when it's within my technical domain, reversible if wrong, and doesn't quietly commit anyone else to something they haven't agreed to — most day-to-day engineering judgment calls fall here, and constantly escalating those just creates bottleneck and signals I don't trust my own judgment, which isn't the senior-level operating model this kind of role expects. The genuinely useful skill is calibrating that line accurately and consistently, and being willing to say 'I made that call, here's why' after the fact rather than needing permission for every reversible decision — autonomy that constantly asks permission isn't really autonomy.
